



P 1

CCA-F PORTFOLIO PROJECT 1 OF 4

Structured Data Extractor

Extract typed invoice data from messy PDFs and images · Measurable accuracy · Public portfolio

DAYS	29–42 of 112 · Weeks 5–6 · 28 hours
MAPS TO	CCA-F Scenario 6 · Domains D4 (heavy) + D5
STACK	Python · anthropic · pydantic · pypdf · typer · rich · pytest
DATA	Public invoice datasets (Hugging Face) · 50-document hand-labeled gold set
REPO	project-01-extractor (separate public GitHub repo)

DAY-BY-DAY BUILD BRIEF · WEEKS 5 – 6

The business problem (one sentence, for your README): Accountants and AP teams spend 3-5 hours per day manually keying invoice data into accounting systems — this extractor processes a PDF or image invoice in under 5 seconds with field-level accuracy measurable against a gold set.

You already know every technical primitive this project needs. The novelty is combining them into one production-grade piece with a real eval harness, a CLI, and a public README a recruiter can understand in 60 seconds.

EXHIBIT 1 • WHAT YOU ALREADY KNOW VS WHAT IS NEW

PATTERN	LEARNED IN	USED IN PROJECT
<code>messages.parse(output_format=Model)</code>	W2 D10–11 + errata	Core extraction engine
Semantic retry loop	W2 D12 + errata	Recovery from validator failures
Prompt caching	W2 D13	Batch cost reduction
Vision API (base64 images)	W1 D3	Image-mode invoice processing
Pydantic <code>model_validator</code>	W2 D12 + errata	Semantic rules (totals must match)
PDF text extraction	NEW Day 29	Text-mode fallback (pypdf)
Confidence-aware routing	NEW Day 33	High/med/low → different pipelines
Hand-labeled gold set + eval harness	NEW Days 30–31	Measurable per-field accuracy
typer CLI + rich output	NEW Day 37	Demo-able interface

EXHIBIT 2 • THE INVOICE SCHEMA (PYDANTIC — SINGLE SOURCE OF TRUTH)

```
from pydantic import BaseModel, Field, model_validator
from typing import Annotated

class LineItem(BaseModel):
    description: Annotated[str, Field(description="Item or service name")]
    quantity: Annotated[float | None, Field(default=None)]
    unit_price: Annotated[float | None, Field(default=None)]
    amount: Annotated[float, Field(description="Line total in invoice currency")]

class Invoice(BaseModel):
    vendor_name: Annotated[str, Field(description="Company that issued the invoice")]
    vendor_address: Annotated[str | None, Field(default=None)]
    invoice_number: Annotated[str, Field(description="Invoice/document number")]
    invoice_date: Annotated[str, Field(description="Issue date, ISO format YYYY-MM-DD if possible")]
```

```
due_date:      Annotated[str | None, Field(default=None)]
line_items:    Annotated[list[LineItem], Field(description="One entry per billed item")]
subtotal:      Annotated[float | None, Field(default=None)]
tax_amount:    Annotated[float | None, Field(default=None)]
total_amount:  Annotated[float, Field(description="Grand total including tax")]
currency:      Annotated[str, Field(description="ISO 4217 code, e.g. USD, EUR, GBP")]
confidence:    Annotated[str, Field(description="high / medium / low based on
legibility")]

@model_validator(mode="after")
def totals_consistent(self):
    # Semantic rule: if subtotal + tax are present, their sum should match total
    if self.subtotal is not None and self.tax_amount is not None:
        expected = round(self.subtotal + self.tax_amount, 2)
        if abs(expected - self.total_amount) > 0.05:
            raise ValueError(
                f"Totals inconsistent: {self.subtotal} + {self.tax_amount}"
                f" = {expected}, but total_amount = {self.total_amount}"
            )
    return self
```

Copy this schema into `src/extractor/schema.py` on Day 29. Do not change it until Day 34 (hardening). The `model_validator` is what the retry loop catches — not JSON parse failures (those can't happen on Sonnet 4.6+) but semantic inconsistencies in the numbers.

EXHIBIT 3 • RECOMMENDED DATA SOURCES

DATASET	FORMAT	BEST FOR	LINK
parsee-ai/invoices-example	PDF + JSON ground truth	First 20 gold-set documents — ground truth already provided	HF
northwind_invocies	PDF	Simple clean PDFs for early testing	HF
katanaml-org/invoices-donut-data-v1	Images (PNG)	Vision-mode testing — scan-style invoices	HF

Download **10 documents each** from parsee-ai and northwind on Day 29. Add 10 image invoices from katanaml later on Day 32 (vision mode day). Hand-label as you go — gold set reaches 50 by mid-Week 5.

DONE WHEN

≥ 90% per-field accuracy on a 50-document gold set · Retry loop demonstrably recovers from injected validator failures · CLI runs end-to-end on a fresh invoice in under 10 seconds · GitHub CI green · README has a demo GIF and an eval results table · Prompt caching visible in the eval cost report.

TOC

14 Build Days

- 29 **W5 Monday** — Repo Setup + Text-Mode Extraction
- 30 **W5 Tuesday** — Gold Set — First 25 Documents
- 31 **W5 Wednesday** — Gold Set Complete + Eval Harness
- 32 **W5 Thursday** — Image Mode
- 33 **W5 Friday** — Confidence Routing
- 34 **W5 Saturday** — Prompt Caching + Cost Report
- 35 **W5 Sunday** — Week 5 Review — Accuracy Push
- 36 **W6 Monday** — Tests
- 37 **W6 Tuesday** — CLI with typer + rich
- 38 **W6 Wednesday** — Accuracy Push to $\geq 90\%$
- 39 **W6 Thursday** — README + Architecture Diagram
- 40 **W6 Friday** — Demo GIF + Quickstart Verification
- 41 **W6 Saturday** — Architecture Decision Records + Stretch Goals
- 42 **W6 Sunday** — Project 1 Ship

Each day has a specific deliverable. The 2-hour block goes entirely to the project — no separate theory labs. Use modern API patterns throughout: no prefix, `messages.parse(output_format=Invoice)` everywhere structured output is needed.

WEEK 05 • P1

Monday

Day 29 of 112

Repo Setup + Text-Mode Extraction

SHIP TODAY A working extractor that reads a PDF as text and returns a validated `Invoice` object.

00-30

Setup (00-30)

Initialize the project repo. From your Desktop:

```
mkdir project-01-extractor
cd project-01-extractor
git init
uv init
uv add anthropic pydantic pypdf typer rich pytest python-dotenv
mkdir -p src/extractor data/raw data/gold tests evals
```

Create `src/extractor/__init__.py` (empty) and `src/extractor/schema.py`. Paste the full `Invoice` + `LineItem` schema from Exhibit 2. This is the single source of truth for the whole project.

Set up `.env` with your API key. Add `.env` and `data/raw/` to `.gitignore`.

Download 10 PDFs from [parsee-ai/invoices-example](#) and 10 from [northwind_invoices](#). Save to `data/raw/`.

30-40

Theory — PDF text vs image mode (10 min)

Two ways to feed an invoice to Claude:

- **Text mode:** extract text with `pypdf`, send as a string. Fast, cheap. Fails on scanned images or complex layouts where `pypdf` loses structure.
- **Image mode:** convert PDF page to a base64 PNG, send as an image content block. Slower, costs more vision tokens. Works on anything Claude can see.

Today: text mode. Day 32: image mode. Day 33: auto-detect and route.

40-110 Lab — text-mode extractor (40-110)

Create `src/extractor/extract.py`:

```
from pathlib import Path
from anthropic import Anthropic
from dotenv import load_dotenv
from pypdf import PdfReader
from .schema import Invoice

load_dotenv()
client = Anthropic()

MAX_RETRIES = 3

SYSTEM = """You extract structured invoice data into JSON.
Be precise with numbers. Use null for missing optional fields.
For confidence: high = all fields clearly visible, medium = some fields unclear,
low = document is blurry, handwritten, or key fields are missing."""

def extract_text(pdf_path: Path) -> str:
    reader = PdfReader(str(pdf_path))
    return "
".join(page.extract_text() or "" for page in reader.pages)

def extract_invoice(pdf_path: Path) -> Invoice:
    text = extract_text(pdf_path)
    if not text.strip():
        raise ValueError(f"No text extracted from {pdf_path.name} – try image mode")

    messages = [{"role": "user", "content": f"Extract invoice data:
{text}"]}
    last_error = None

    for attempt in range(1, MAX_RETRIES + 1):
        response = client.messages.parse(
            model="claude-sonnet-4-6",
            max_tokens=1024,
            system=SYSTEM,
            messages=messages,
            output_format=Invoice,
        )
        try:
            return response.parsed_output
        except ValueError as e:
            last_error = e
            if attempt == MAX_RETRIES:
                break
            messages = [
                {"role": "user", "content": f"Extract invoice data:
{text}"}],
```

```
        {"role": "user", "content": (
            f"Previous extraction failed validation: {e}. "
            "Re-extract, paying close attention to that constraint."
        )},
    ],

    from .schema import ExtractionError
    raise ExtractionError(f"Failed after {MAX_RETRIES} attempts: {last_error}")
```

Add to `schema.py`: `class ExtractionError(Exception): pass`

Write a quick smoke test in a scratch script — pick one PDF, call `extract_invoice()`, print the result. Make sure it runs before committing.

110-120 Capture (110-120)

Commit everything. Message: `day 29: text-mode extractor working`. Push to GitHub (create the public repo now if you haven't).

COMMITTED BY 10PM `extract_invoice()` returns a validated `Invoice` on at least 5 of your 20 PDFs without crashing.

WEEK 05 • P1

Tuesday

Day 30 of 112

Gold Set — First 25 Documents

SHIP TODAY 25 hand-labeled invoice records in `evals/gold.jsonl`. This is your most important asset.

00-20 Theory — why the gold set IS the project (00-20)

Every improvement you make from Day 31 onward will be measured against this gold set. Without it, "my extractor works" is a feeling. With it, "my extractor achieves 94% field accuracy on 50 documents" is a number you put in your README and defend in an interview.

Labeling is tedious. It is also the single most important thing you do in Week 5. No shortcuts.

Gold set format — each line of `evals/gold.jsonl` is:

```
{"file": "inv_001.pdf", "invoice_number": "INV-2024-001", "vendor_name": "Acme Corp",  
  "invoice_date": "2024-01-15", "total_amount": 1250.00, "currency": "USD",  
  "line_items": [{"description": "Consulting services", "amount": 1250.00}],  
  "subtotal": 1250.00, "tax_amount": 0.0, "confidence": "high"}
```

You only need the fields that are cleanly visible. A field genuinely missing from the document should be `null` in your gold record — that's the correct label, not a gap.

20-110 Lab — label 25 documents (20-110)

Open each PDF. Read it. Type the ground truth into a `.jsonl` line. That's it.

Tips that save time:

- Write a small helper script that pretty-prints your extractor's output for a file. Compare to the PDF. Mark where it's wrong. That's faster than labeling from scratch.
- Prioritize variety: different vendors, different layouts, some with tax, some without, at least 3 multi-line-item invoices.
- Label `confidence` accurately — it's not a quality judgment of your extractor, it's a property of the document (blurry, handwritten, or missing fields = low).

Target: 25 labeled documents today. The other 25 come on Day 31.

110-120 Capture (110-120)

Commit `evals/gold.jsonl` with 25 entries. Message: `day 30: 25 gold labels`.

COMMITTED BY 10PM 25 hand-labeled records committed. No automate-the-labeling shortcuts.

WEEK 05 • P1

Wednesday

Day 31 of 112

Gold Set Complete + Eval Harness

SHIP TODAY 50 labeled documents and a working eval script that scores per-field accuracy.

00-50 Gold set — 25 more (00-50)

Same as yesterday. Label 25 more documents. Focus on edge cases:

- Invoices with multiple pages
- At least 5 with line items that have quantity × unit_price (verify your `model_validator` triggers correctly on bad totals)
- At least 3 where the vendor address spans multiple lines
- At least 2 non-USD currencies

50 total by end of this block.

50-110 Lab — eval harness (50-110)

Create `evals/run_eval.py`:

```
import json
import time
from pathlib import Path
from src.extractor.extract import extract_invoice
from src.extractor.schema import ExtractionError

GOLD_FILE = Path("evals/gold.jsonl")
DATA_DIR = Path("data/raw")

SCORED_FIELDS = [
    "invoice_number", "vendor_name", "invoice_date",
    "total_amount", "currency",
]

def score_field(predicted, gold_val, field: str) -> bool:
    if gold_val is None:
        return True # missing in gold means "not applicable"
    if predicted is None:
        return False
    # Normalize strings
    p = str(predicted).strip().lower()
    g = str(gold_val).strip().lower()
    if field == "total_amount":
        try:
            return abs(float(p) - float(g)) < 0.05
        except (ValueError, TypeError):
            return False
    return p == g

def run():
    gold = [json.loads(l) for l in GOLD_FILE.read_text().splitlines() if l.strip()]
    results = {f: {"correct": 0, "total": 0} for f in SCORED_FIELDS}
    errors = []
    t0 = time.time()

    for item in gold:
        path = DATA_DIR / item["file"]
        try:
            inv = extract_invoice(path)
            pred = inv.model_dump()
        except ExtractionError as e:
            errors.append((item["file"], str(e)))
            continue

        for field in SCORED_FIELDS:
            gold_val = item.get(field)
            if gold_val is None:
                continue
            results[field]["total"] += 1
            if score_field(pred.get(field), gold_val, field):
                results[field]["correct"] += 1
```

```

    elapsed = time.time() - t0
    print(f"
Eval results ({len(gold)} documents, {elapsed:.1f}s):
")
    total_correct = total_scored = 0
    for field, r in results.items():
        if r["total"] == 0:
            continue
        acc = r["correct"] / r["total"] * 100
        total_correct += r["correct"]
        total_scored += r["total"]
        print(f"  {field:<20} {acc:5.1f}%  ({r['correct']}/{r['total']})")
    if total_scored:
        print(f"
OVERALL: {total_correct/total_scored*100:.1f}%")
    if errors:
        print(f"
Extraction errors: {len(errors)}")
        for f, e in errors[:5]:
            print(f"    {f}: {e}")

if __name__ == "__main__":
    run()

```

Run it:

```
uv run python evals/run_eval.py
```

You will likely score somewhere between 60-80% on first run. That's fine. The number is a baseline. Write it in `evals/history.md` with today's date and prompt version. You'll improve it through Day 35.

110-120 Capture (110-120)

Commit gold set (50 entries) + eval harness. Log baseline accuracy in `evals/history.md`. Message: `day 31: gold set complete, baseline eval`.

COMMITTED BY 10PM 50-document gold set complete. Eval harness running. Baseline accuracy logged.

WEEK 05 • P1

Thursday

Day 32 of 112

Image Mode

SHIP TODAY *The extractor works on scanned invoices and poor-quality PDFs via the vision API.*

00-15 Theory — when text mode fails (00-15)

pypdf extracts text that was digitally embedded in the PDF. Scanned documents, photos of invoices, and some export-from-Word PDFs produce garbled or empty text. For these, you send the page as an image.

The pattern is the same as Week 1 Day 3 (vision lab), with one addition: PDF → image conversion using `pypdf`'s page rendering or the `pdf2image` library.

Today you also add auto-detection: try text first; fall back to image mode if the extracted text is too short to be a real invoice.

15-110 Lab — image-mode extractor (15-110)

Add to your project:

```
uv add pdf2image pillow
```

Create `src/extractor/extract_image.py`:

```
import base64
from pathlib import Path
from anthropic import Anthropic
from dotenv import load_dotenv
from pdf2image import convert_from_path
from .schema import Invoice, ExtractionError

load_dotenv()
client = Anthropic()

MAX_RETRIES = 3

SYSTEM = """You extract structured invoice data from an image of an invoice.
Be precise with numbers. Use null for missing optional fields.
For confidence: high = all fields clearly visible, medium = some unclear,
low = blurry, handwritten, or key fields are missing."""

def pdf_to_base64_images(pdf_path: Path) -> list[str]:
    """Convert PDF pages to base64 PNG strings."""
    images = convert_from_path(str(pdf_path), dpi=150)
    result = []
    for img in images:
        import io
        buf = io.BytesIO()
        img.save(buf, format="PNG")
        result.append(base64.standard_b64encode(buf.getvalue()).decode())
    return result

def extract_invoice_image(pdf_path: Path) -> Invoice:
    images = pdf_to_base64_images(pdf_path)
    # Use only first 2 pages – most invoices fit in 1-2 pages
    content = []
    for b64 in images[:2]:
        content.append({
            "type": "image",
            "source": {"type": "base64", "media_type": "image/png", "data": b64},
        })
    content.append({"type": "text", "text": "Extract the invoice data from this image."})

    messages = [{"role": "user", "content": content}]
    last_error = None

    for attempt in range(1, MAX_RETRIES + 1):
        response = client.messages.parse(
            model="claude-sonnet-4-6",
```

```

        max_tokens=1024,
        system=SYSTEM,
        messages=messages,
        output_format=Invoice,
    )
    try:
        return response.parsed_output
    except ValueError as e:
        last_error = e
        if attempt == MAX_RETRIES:
            break
        messages.append({
            "role": "user",
            "content": f"Extraction failed validation: {e}. Re-extract carefully.",
        })

    raise ExtractionError(f"Image mode failed after {MAX_RETRIES} attempts: {last_error}")

```

Update `extract.py` to add auto-routing:

```

from .extract_image import extract_invoice_image

MIN_TEXT_CHARS = 100 # below this, text extraction failed

def extract_invoice(pdf_path: Path, force_mode: str = "auto") -> Invoice:
    """
    force_mode: "auto" | "text" | "image"
    auto: tries text first, falls back to image if text is too short.
    """
    if force_mode == "image":
        return extract_invoice_image(pdf_path)

    text = extract_text(pdf_path)

    if force_mode == "text" or len(text.strip()) >= MIN_TEXT_CHARS:
        # ... existing text-mode logic ...
        pass

    # Text too short – fall back to image mode
    return extract_invoice_image(pdf_path)

```

Download 10 image invoices from katanaml-org/invoices-donut-data-v1 to `data/raw/`. Run image mode on 3 of them manually. Check the output. Add 5 of the image invoices to your gold set (reaching ~55 total).

110-120 Capture (110-120)

Commit image mode + auto-routing. Update eval harness to handle the new image-invoice additions. Message: `day 32: image mode + auto-routing`.

COMMITTED BY 10PM Image-mode extraction working on scanned invoices. Auto-routing logic commits.

WEEK 05 • P1

Friday

Day 33 of 112

Confidence Routing

SHIP TODAY *High-confidence invoices go to output. Medium and low route to different pipelines. The system never silently drops a bad extraction.*

00-15 Theory — confidence as a first-class field (00-15)

Your schema already has a `confidence` field (`Literal["high", "medium", "low"]`). Today you act on it:

- **high** — write to `output/approved/` and count as done.
- **medium** — write to `output/review/` with a flag file. A human should verify before downstream use.
- **low** — write to `output/rejected/` with the reason. Try image mode if text mode was used. Log for analysis.

This pattern is exactly what the CCA-F exam tests under "escalation" — you know when to route to human review rather than silently proceeding. It's also the most impressive thing to mention in a portfolio README.

15-110 Lab — routing pipeline (15-110)Create `src/extractor/pipeline.py`:

```

import json
from pathlib import Path
from .extract import extract_invoice
from .schema import Invoice, ExtractionError

OUTPUT_DIRS = {
    "high": Path("output/approved"),
    "medium": Path("output/review"),
    "low": Path("output/rejected"),
}

def process_invoice(pdf_path: Path, verbose: bool = False) -> dict:
    """
    Process one invoice and route based on confidence.
    Returns a result dict with keys: file, status, confidence, output_path, error.
    """
    for d in OUTPUT_DIRS.values():
        d.mkdir(parents=True, exist_ok=True)

    result = {
        "file": pdf_path.name,
        "status": None,
        "confidence": None,
        "output_path": None,
        "error": None,
    }

    try:
        invoice = extract_invoice(pdf_path)
        result["confidence"] = invoice.confidence

        out_dir = OUTPUT_DIRS[invoice.confidence]
        out_path = out_dir / pdf_path.with_suffix(".json").name
        out_path.write_text(invoice.model_dump_json(indent=2))
        result["output_path"] = str(out_path)
        result["status"] = "ok"

        if verbose:
            print(f" [{invoice.confidence.upper()}] {pdf_path.name} -> {out_path}")

    except ExtractionError as e:
        result["status"] = "error"
        result["error"] = str(e)
        err_path = OUTPUT_DIRS["low"] / pdf_path.with_suffix(".error.txt").name
        err_path.write_text(str(e))
        if verbose:
            print(f" [ERROR] {pdf_path.name}: {e}")

    return result

```



```
def process_batch(pdf_dir: Path, verbose: bool = True) -> list[dict]:  
    pdfs = sorted(pdf_dir.glob("*.pdf"))  
    return [process_invoice(p, verbose=verbose) for p in pdfs]
```

Run the pipeline on your full `data/raw/` folder:

```
uv run python -c "from src.extractor.pipeline import process_batch; from pathlib import Path;  
r=process_batch(Path('data/raw'), verbose=True); print(f'{len(r)} processed')"
```

Look at the distribution across `output/approved/`, `output/review/`, `output/rejected/`. If everything is "high", your model is overconfident. Check a few medium and low ones manually — the confidence label should reflect actual document quality.

110–120 Capture (110–120)

Commit routing pipeline. Note the approved/review/rejected distribution in `evals/history.md`. Message: `day 33: confidence routing pipeline`.

COMMITTED BY 10PM Three-way routing working. No extraction silently disappears.

WEEK 05 • P1

Saturday

Day 34 of 112

Prompt Caching + Cost Report

SHIP TODAY *The system prompt plus schema is cached. Eval report shows cost per document with and without cache.*

00-15

Theory (00-15)

Your system prompt (`SYSTEM` constant) plus the Pydantic schema that `messages.parse()` derives from your model is sent on every extraction call. It's static across the whole batch. Classic cache candidate.

You already know the pattern from Week 2 Day 13. Today you apply it to the extractor and measure the savings in the eval report.

15-110 Lab — add caching, measure savings (15-110)

Update `extract.py` — wrap the system prompt in a cache block:

```
# Replace the plain SYSTEM string with a list of content blocks
SYSTEM_CACHED = [
    {
        "type": "text",
        "text": SYSTEM,
        "cache_control": {"type": "ephemeral"},
    }
]

# Use SYSTEM_CACHED in your messages.parse() call instead of SYSTEM
response = client.messages.parse(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    system=SYSTEM_CACHED,
    messages=messages,
    output_format=Invoice,
)
```

Update `run_eval.py` to track and report token usage per call:

```
# In your eval loop, accumulate:
total_input = 0
total_cache_read = 0
total_cache_create = 0

# After each extract_invoice() call, the usage is in response.usage
# You'll need to thread usage out of extract.py – simplest: return (invoice, usage) tuple

# At the end of eval:
input_cost = total_input / 1_000_000 * 3.0
cache_cost = total_cache_read / 1_000_000 * 0.30 # 10x cheaper
create_cost = total_cache_create / 1_000_000 * 3.75
print(f"
Cost breakdown:")
print(f"  Uncached input:   ${input_cost:.4f}")
print(f"  Cache reads:       ${cache_cost:.4f}")
print(f"  Cache writes:      ${create_cost:.4f}")
print(f"  Est. total:        ${input_cost + cache_cost + create_cost:.4f}")
print(f"  Cost per doc:      ${((input_cost + cache_cost + create_cost)/len(gold)):.4f}")
```

Run the eval twice — first run pays to write the cache, subsequent runs read it cheaply. Log both in `evals/history.md`.

110-120 Capture (110-120)

Commit. Log cost-per-document before and after caching in `history.md`. Message:

day 34: prompt caching, cost report.

COMMITTED BY 10PM Cache visible in eval stats. Cost-per-document logged. Cache reduces input cost by ≥80%.

WEEK 05 • P1

Sunday

Day 35 of 112

Week 5 Review — Accuracy Push

SHIP TODAY Accuracy at or above 85% overall before starting Week 6.**00-100** Review + accuracy push (00-100)

Run the full eval. Read the misses carefully — every wrong field is a prompt-engineering clue. Common patterns and fixes:

- **invoice_number wrong** — add field description: "The invoice/PO number, typically starts with INV- or # or is a sequence of digits near the top of the document."
- **total_amount off by tax** — your `model_validator` should be catching this. If it's not triggering, check that subtotal and tax are being extracted.
- **date format wrong** — strengthen the field description: "Convert to ISO YYYY-MM-DD. If only month/year visible, use YYYY-MM-01."
- **currency missing** — add to system prompt: "If no currency symbol is present, infer from vendor country or default to USD."
- **line items incomplete** — add to system prompt: "Extract ALL line items. Do not summarize. If a line has no price, still include it with amount: 0."

Each change: re-run eval, check if it moved the number, log in history.md. This prompt-tuning loop IS the job for today.

Target: ≥85% overall before you close the laptop today.

100-110 Capture (100-120)

Commit improved system prompt + updated history.md. Message: `week 5 complete: eval at X%`. Replace X with your actual number. Push.

COMMITTED BY 10PM ≥85% overall accuracy logged in `evals/history.md`. Week 5 complete.

WEEK 06 • P1

Monday

Day 36 of 112

Tests

SHIP TODAY *A pytest suite that will keep working as you refactor. Green CI by end of day.*

00-15

Theory (00-15)

Tests serve two purposes here: they protect your eval score while you refactor in Days 37-40, and they demonstrate engineering hygiene to recruiters. A portfolio project with 0 tests looks unfinished. A project with 15 tests, a coverage badge, and green CI looks professional.

Focus on four test categories:

- **Schema tests** — does your validator trigger correctly?
- **Routing tests** — does confidence route to the right folder?
- **Retry tests** — does the loop retry on validator failure?
- **Regression tests** — do your 5 "known good" invoices still extract correctly?

15-110 Lab — write the test suite (15-110)

Create `tests/test_schema.py`:

```
import pytest
from pydantic import ValidationError
from src.extractor.schema import Invoice, LineItem

def make_valid_invoice(**overrides):
    base = dict(
        vendor_name="Acme Corp",
        invoice_number="INV-001",
        invoice_date="2024-01-15",
        line_items=[LineItem(description="Services", amount=100.0)],
        total_amount=107.0,
        currency="USD",
        subtotal=100.0,
        tax_amount=7.0,
        confidence="high",
    )
    base.update(overrides)
    return Invoice(**base)

def test_valid_invoice_passes():
    inv = make_valid_invoice()
    assert inv.vendor_name == "Acme Corp"

def test_totals_mismatch_raises():
    with pytest.raises(ValidationError, match="inconsistent"):
        make_valid_invoice(subtotal=100.0, tax_amount=7.0, total_amount=999.0)

def test_totals_within_tolerance_passes():
    # Rounding errors up to 5 cents are OK
    inv = make_valid_invoice(subtotal=100.0, tax_amount=7.0, total_amount=107.04)
    assert inv.total_amount == 107.04

def test_null_subtotal_skips_check():
    # If subtotal is None, we can't validate so we skip
    inv = make_valid_invoice(subtotal=None, tax_amount=None, total_amount=500.0)
    assert inv.total_amount == 500.0

def test_confidence_must_be_valid():
    with pytest.raises(ValidationError):
        make_valid_invoice(confidence="excellent")
```

Create `tests/test_pipeline.py` with mocked extraction (use `unittest.mock.patch` to avoid real API calls in tests).

Create `tests/test_routing.py` that verifies the three output folders are created correctly.

Set up GitHub Actions CI. Create `.github/workflows/ci.yml`:

```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
```

```
steps:
```

- uses: actions/checkout@v4
- uses: astral-sh/setup-uv@v3
- run: uv sync
- run: uv run pytest tests/ -v

110-120 Capture (110-120)

Commit tests + CI config. Push. Verify the green checkmark appears on GitHub. Message: `day 36: test suite + CI`.

COMMITTED BY 10PM pytest suite passing locally. GitHub Actions CI green on push.

WEEK 06 • P1

Tuesday

Day 37 of 112

CLI with typer + rich

SHIP TODAY *A polished command-line interface. The demo you'll record for the README GIF.*

00-10 Theory (00-10)

typer builds a full CLI from Python type annotations. rich adds colors, progress bars, and formatted tables. Together they make your extractor look like a professional tool rather than a research script. The CLI is what you'll screen-record for the demo GIF.

Skim [typer docs](#) (CLI basics) and [rich docs](#) (table and progress bar examples). 10 minutes.

10-110 Lab — build the CLI (10-110)Create `src/extractor/cli.py`:

```

import json
from pathlib import Path
import typer
from rich.console import Console
from rich.table import Table
from rich.progress import track

from .pipeline import process_invoice, process_batch
from .schema import Invoice

app = typer.Typer(help="Invoice Data Extractor – CCA-F Portfolio Project 1")
console = Console()

@app.command()
def extract(
    file: Path = typer.Argument(..., help="PDF invoice to extract"),
    mode: str = typer.Option("auto", help="Extraction mode: auto / text / image"),
    json_out: bool = typer.Option(False, "--json", help="Output raw JSON"),
):
    """Extract data from a single invoice."""
    result = process_invoice(file, verbose=False)
    if result["status"] == "error":
        console.print(f"[red]Extraction failed:[/red] {result['error']}")
        raise typer.Exit(1)

    data = json.loads(Path(result["output_path"]).read_text())
    if json_out:
        console.print_json(json.dumps(data))
        return

    # Pretty table
    t = Table(title=f"{file.name}", show_header=True)
    t.add_column("Field", style="bold cyan")
    t.add_column("Value")
    for k, v in data.items():
        if k == "line_items":
            t.add_row(k, f"{len(v)} item(s)")
        else:
            t.add_row(k, str(v) if v is not None else "[dim]null[/dim]")
    console.print(t)
    conf_color = {"high": "green", "medium": "yellow", "low": "red"}
    c = data.get("confidence", "low")
    console.print(f"Confidence: [{conf_color.get(c, 'white')}] {c.upper()}[/]")

@app.command()
def batch(
    directory: Path = typer.Argument(..., help="Directory of PDF invoices"),
    verbose: bool = typer.Option(True),

```

```

):
    """Process all PDFs in a directory."""
    pdfs = sorted(directory.glob("*.pdf"))
    console.print(f"[bold]Processing {len(pdfs)} invoices...[/bold]")

    results = []
    for pdf in track(pdfs, description="Extracting..."):
        results.append(process_invoice(pdf, verbose=False))

    ok = sum(1 for r in results if r["status"] == "ok")
    errs = len(results) - ok
    by_conf = {}
    for r in results:
        c = r.get("confidence") or "error"
        by_conf[c] = by_conf.get(c, 0) + 1

    console.print(f"
[green]{ok}[/green] succeeded, [red]{errs}[/red] failed")
    for conf, count in sorted(by_conf.items()):
        console.print(f"  {conf}: {count}")

@app.command()
def eval_run():
    """Run the full evaluation against the gold set."""
    import subprocess, sys
    subprocess.run([sys.executable, "-m", "evals.run_eval"], check=True)

if __name__ == "__main__":
    app()

```

Add entrypoint to `pyproject.toml` :

```

[project.scripts]
invoice-extract = "src.extractor.cli:app"

```

Test all three commands:

```

uv run invoice-extract extract data/raw/inv_001.pdf
uv run invoice-extract extract data/raw/inv_001.pdf --json
uv run invoice-extract batch data/raw/

```

110-120 Capture (110-120)

Commit CLI. Message: `day 37: typer CLI + rich output` . All tests still pass.

COMMITTED BY 10PM Three-command CLI working. Rich table output looks portfolio-ready.

WEEK 06 • P1

Wednesday

Day 38 of 112

Accuracy Push to ≥90%

SHIP TODAY Final prompt tuning session. Target: ≥90% overall. Log the before/after in `history.md`.

00-100 Full eval + prompt tuning (00-100)

Run `uv run invoice-extract eval-run`. Read every miss. The pattern from Day 35 applies — but now you have more data and a cleaner system to observe.

If you're already at ≥90%, spend today on:

- Adding few-shot examples to your system prompt (2-3 complete invoice examples inline)
- Measuring whether few-shot moves accuracy vs just the schema + descriptions
- Checking whether image mode scores differently from text mode (separate eval by mode)

If you're below 85%, prioritize the top-missing field before anything else. One field improvement typically raises overall accuracy 3-5 points.

Whatever you do: every prompt change gets its own run logged in `evals/history.md`. This is your paper trail. The README table comes from this file.

100-120 Capture (100-120)

Commit final prompt. Log final accuracy in `history.md`. Message: `day 38: accuracy push, final eval at X%`.

COMMITTED BY 10PM ≥90% overall accuracy. `history.md` shows the improvement curve.

WEEK 06 • P1

Thursday

Day 39 of 112

README + Architecture Diagram

SHIP TODAY A README that a recruiter can scan in 60 seconds and understand the project's value.

00-15 Theory — the README formula (00-15)

Recruiters skim. Your README must survive a 60-second skim and communicate value. Use this exact structure, in this order:

1. **Problem** (one sentence): "Accountants spend X hours/day keying invoices manually; this extracts to typed JSON in under 5 seconds."
2. **Architecture diagram** (the visual that explains the system)
3. **Quickstart** (three commands that work on a fresh machine)
4. **Eval results table** (field-level accuracy, cost-per-doc, with/without cache)
5. **Trade-offs and limits** (what it does badly and why)
6. **What I'd do next** (shows engineering judgment)

15-110 Lab — write README + diagram (15-110)

Create the architecture diagram in Excalidraw (free at excalidraw.com) or draw.io. Export as PNG to `docs/architecture.png`. The diagram should show:

- PDF/Image input → Auto-detect (text or image mode) → Claude API
- Claude API → Invoice schema → `model_validator`
- Validator pass → Confidence routing (3 paths)
- Retry loop arrow on validator failure

Write `README.md`. The eval results table should look like:

Field	Accuracy	Notes
invoice_number	96%	
vendor_name	98%	
invoice_date	91%	format variance
total_amount	94%	
currency	99%	
Overall	**95%**	50-doc gold set

Metric	Value
Cost per doc (cached)	\$0.003
Cost per doc (no cache)	\$0.021
Cache savings	86%
Avg extraction time	3.2s

Replace numbers with your actual results from `evals/history.md`.

110-120 Capture (110-120)

Commit README + diagram. Message: `day 39: README + architecture diagram`.

COMMITTED BY 10PM README follows the 6-section formula. Diagram committed. Eval table uses real numbers.

WEEK 06 • P1

Friday

Day 40 of 112

Demo GIF + Quickstart Verification

SHIP TODAY A 30-second screen recording proving the CLI works. Quickstart verified on a clean environment.

00-80 Lab — record the demo (00-80)

Record a GIF or short video (30 seconds max) showing:

1. Running `invoice-extract extract some_invoice.pdf`
2. The rich table appearing with all fields populated
3. The confidence indicator at the bottom
4. Optionally: running batch mode on a small folder

Tools: LICEcap (free, Windows), ScreenToGif (free, Windows), or any screen recorder that outputs GIF/MP4. Save as `docs/demo.gif` and link it at the top of your README above the architecture diagram.

Verify the quickstart: create a fresh folder, clone your repo, and try to run the three quickstart commands exactly as written. If anything breaks, fix the README. "It works on my machine" is not a public portfolio.

80-110 Polish pass (80-110)

Read your README as a stranger who has never heard of you. Does it communicate value in 60 seconds? Fix anything that requires prior knowledge to understand.

Check: does your repo have a license file (MIT is fine), a `.gitignore` covering `.env` and your raw invoice data, and a clear "this is a portfolio project, not production software" note in the README footer? All three should be yes.

110-120 Capture (110-120)

Commit demo GIF + any README fixes. Message: `day 40: demo GIF + quickstart verified`. All CI tests green.

COMMITTED BY 10PM Demo GIF committed. Quickstart works on a fresh clone. CI green.

WEEK 06 • P1

Saturday

Day 41 of 112

Architecture Decision Records + Stretch Goals

SHIP TODAY Document one non-obvious trade-off. If time remains, add one stretch feature.

00-40 ADR (00-40)

Create `docs/adr/0001-text-vs-image-routing.md`. An Architecture Decision Record explains a non-obvious choice. Template:

```
# ADR-0001: Automatic Text/Image Mode Routing

## Status
Accepted

## Context
Invoices arrive as both digital PDFs (text-extractable) and scanned images.
Two extraction modes exist: text (fast, cheap) and image (slower, more expensive but
works on scans). The system needs to choose automatically.

## Decision
Attempt text extraction first. If the extracted text is below 100 characters, fall back
to image mode. Threshold is configurable via MIN_TEXT_CHARS.

## Consequences
- Positive: most digital PDFs are processed cheaply via text mode.
- Negative: some complex layouts (multi-column, tables) produce garbled text above
  the threshold, causing text mode to run on bad input. Known limitation.
- Alternative considered: always use image mode. Rejected: 5x cost increase on
  the majority of clean PDFs.
```

ADRs show engineering judgment. They're what distinguish "I built something" from "I thought about trade-offs." One ADR is enough for this project.

40-110 Stretch goals — pick one (40-110)

Pick the one most relevant to your situation. Truly optional — the project is shippable without any of these.

- **Streamlit web UI** — file-upload form, show extracted fields side-by-side with the PDF viewer. Most visually impressive for a portfolio.
- **LLM-as-judge eval** — a second Claude call that scores whether the extraction "looks right" for soft fields like `vendor_address`. Shows you know about model-based grading (Skilljar Week 2 topic).
- **Batch cost estimator CLI command** — `invoice-extract estimate ./invoices/` prints projected cost before running. Useful + demonstrates your cost-awareness habit from Week 1.
- **Per-mode accuracy split** — separate eval results for text-mode vs image-mode documents. Shows analytical rigor.

110-120 Capture (110-120)

Commit ADR + any stretch work. Message: `day 41: ADR + [stretch feature if applicable]`.

COMMITTED BY 10PM ADR committed. Stretch goal (if chosen) working and committed.

WEEK 06 • P1

Sunday

Day 42 of 112

Project 1 Ship

SHIP TODAY *Everything committed, CI green, README polished. Announce on LinkedIn.***00-80** **Final checklist (00-80)**

Work through this list in order. Don't skip items.

1. Run `uv run pytest tests/ -v`. All green.
2. Run `uv run invoice-extract eval-run`. Accuracy $\geq 90\%$. If not, one more prompt-tuning pass before anything else.
3. Run `uv run invoice-extract batch data/raw/`. No crashes.
4. Read README top to bottom as a stranger. Fix any jargon or assumed context.
5. Verify demo GIF still reflects the current CLI (commands haven't changed).
6. Check `.gitignore`: `data/raw/` not committed, `.env` not committed.
7. Verify the repo is public on GitHub.
8. Add "CCA-F Portfolio Project 1" to the repo description and topics.

80-110 **Announce (80-110)**

Write a LinkedIn post (3-5 sentences):

- What it does (problem → solution)
- The most interesting technical challenge (confidence routing? the retry loop? caching reducing cost by 86%?)
- Link to the GitHub repo
- Something you'd do differently if starting over

Don't overthink it. Ship the post. Engagement from engineers who've done similar work is more valuable feedback than any eval metric.

110-120 **Capture (110-120)**

Final commit: `project 1 complete`. Note the LinkedIn post URL in `evals/history.md`. Close the laptop. Week 7 (agents) starts tomorrow.

COMMITTED BY 10PMPublic repo, CI green, $\geq 90\%$ accuracy, LinkedIn post live.